USEI13 - Análise de Complexidade Temporal

Contexto

As operações ferroviárias dependem de um pequeno conjunto de estações hub que concentram transferências, conectam regiões e atuam como pontos críticos. Identificar quantitativamente estes hubs ajuda no planeamento de capacidade, robustez (onde falhas têm impacto significativo) e priorização de melhorias.

User Story

> **Como planeador de manutenção**, quero calcular medidas de centralidade para cada estação, tais como betweenness (intermediação em caminhos mais curtos), harmonic closeness (acessibilidade) e strength/degree (conectividade direta), e combiná-las num HubScore composto (normalizado e ponderado), para que eu possa classificar e exportar os hubs da rede para tomada de decisão. Todos os scores são normalizados para [0,1] por componente.
>
> hubscore = 0.35 * betw + 0.35 * harmonic closeness + 0.30 * strengthNorm

---

Análise da Complexidade Temporal

Variáveis de Análise

- **n**: Número de vértices (estações)
- **m**: Número de arestas (linhas/conexões)

Método Principal

RailHubAnalysisResult analyzeNetwork()

Linha	Código	Complexidade Temporal	Justificação
45	`Graph<Station, Weight> graph = repository.getUndirectedGraph();`	**O(1)**	Acesso direto ao grafo já construído

Linha	Código	Complexidade Temporal	Justificação
46	`if (graph == null || graph.numVertices() == 0)`	**O(1)**	Verificação de referência e acesso a campo
49	`if (graph.numEdges() == 0)`	**O(1)**	Acesso a campo
53	`ArrayList<Station> vertices = graph.vertices();`	**O(n)**	Cria nova ArrayList copiando todos os vértices
54	`Map<Integer, NodeMetrics> metrics = initMetrics(vertices);`	**O(n)**	Ver análise do método `initMetrics()`
56	`computeDegreeAndStrength(graph, metrics);`	**O(m)**	Ver análise do método `computeDegreeAndStrength()`
57	`computeCentralities(graph, vertices, metrics);`	**O(n·m log n + n^2)**	Ver análise do método `computeCentralities()`
59	`List<StationHubScore> scores = buildScores(vertices, metrics);`	**O(n)**	Ver análise do método `buildScores()`
60	`scores.sort(Comparator.comparingDouble(...).reversed());`	**O(n log n)**	Ordenação usando TimSort (merge sort híbrido)
65	`return new RailHubAnalysisResult(scores, complexityNote);`	**O(1)**	Criação de objeto

Nota: A complexidade temporal dos métodos auxiliares é explorada nas tabelas de métodos auxiliares abaixo.

Complexidade Temporal Total:

.: $O(1 + 1 + 1 + n + n + m + n{\cdot}m \log n + n^2 + n + n \log n + 1) = \mathbf{O(n{\cdot}m \log n + n^2)}$

Métodos Auxiliares

Map<Integer, NodeMetrics> initMetrics(Collection stations)

Linha	Código	Complexidade Temporal	Justificação
69	`Map<Integer, NodeMetrics> metrics = new HashMap<>();`	**O(1)**	Criação de HashMap vazio

Linha	Código	Complexidade Temporal	Justificação
70-72	`for (Station station : stations) {`	**O(n)**	Itera sobre todos os vértices
71	`metrics.put(station.getStationID(), new NodeMetrics());`	**O(1)**	Inserção em HashMap (amortizado)

Complexidade Temporal Total:

.: O(1 + n·1) = **O(n)**

---

void computeDegreeAndStrength(Graph<Station, Weight> graph, Map<Integer, NodeMetrics> metrics)

Linha	Código	Complexidade Temporal	Justificação
77	`Set<Integer> knownIds = metrics.keySet();`	**O(1)**	Retorna view do conjunto de chaves
78	`Map<Integer, Set<Integer>> neighbours = new HashMap<>();`	**O(1)**	Criação de HashMap
79-81	`for (Integer id : knownIds) {`	**O(n)**	Itera sobre todos os vértices
80	`neighbours.put(id, new HashSet<>());`	**O(1)**	Inserção em HashMap (amortizado)
83	`Set<String> seenEdges = new HashSet<>();`	**O(1)**	Criação de HashSet
84	`for (Edge<Station, Weight> edge : graph.edges()) {`	**O(m)**	Itera sobre todas as arestas
85	`if (edge == null) continue;`	**O(1)**	Verificação de referência
86-87	Acesso a `getVOrig()` e `getVDest()`	**O(1)**	Acesso a campos
90-91	Acesso a `getStationID()`	**O(1)**	Acesso a campos
92-93	`metrics.get(depId)` e `metrics.get(arrId)`	**O(1)**	Lookup em HashMap (amortizado)
96	Construção de String `key`	**O(1)**	Concatenação de strings pequenas

---

Linha	Código	Complexidade Temporal	Justificação
97	`if (seenEdges.add(key))`	**O(1)**	Inserção em HashSet (amortizado)
98	`if (neighbours.get(depId).add(arrId))`	**O(1)**	Lookup e inserção (amortizado)
99	`depMetrics.degree++;`	**O(1)**	Incremento
101	`if (neighbours.get(arrId).add(depId))`	**O(1)**	Lookup e inserção (amortizado)
102	`arrMetrics.degree++;`	**O(1)**	Incremento
105	`edge.getWeight().getCapacity()`	**O(1)**	Acesso a campos
106-107	Operações aritméticas de soma	**O(1)**	Soma

Complexidade Temporal Total:

.: $O(1 + 1 + n \cdot 1 + 1 + m \cdot 1) = O(n + m) =$ **O(m)**

---

void computeCentralities(Graph<Station, Weight> graph, ArrayList vertices, Map<Integer, NodeMetrics> metrics)

Linha	Código	Complexidade Temporal	Justificação
114	`int n = vertices.size();`	**O(1)**	Acesso a campo size
115	`if (n == 0) return;`	**O(1)**	Verificação condicional
119	`Map<Station, Integer> indexOf = buildVertexIndexMap(vertices);`	**O(n)**	Ver análise do método `buildVertexIndexMap()`
120	`double[] betweenness = new double[n];`	**O(n)**	Inicialização de array
121	`double[] harmonic = new double[n];`	**O(n)**	Inicialização de array
123-125	`for (int s = 0; s < n; s++) {`	**O(n)**	Itera sobre todos os vértices como origem
124	`computeSingleSourceCentralities(...);`	**O(m log n + n)**	Ver análise do método `computeSingleSourceCentralities()`

Linha	Código	Complexidade Temporal	Justificação
127	`updateMetricsFromArrays(vertices, metrics, betweenness, harmonic);`	**O(n)**	Ver análise do método `updateMetricsFromArrays()`
128	`normalizeAndComputeHubScore(vertices, metrics, n);`	**O(n)**	Ver análise do método `normalizeAndComputeHubScore()`

Complexidade Temporal Total:

.: $O(1 + 1 + n + n + n + n \cdot (m \log n + n) + n + n) = \mathbf{O(n \cdot m \log n + n^2)}$

Map<Station, Integer> buildVertexIndexMap(ArrayList vertices)

Linha	Código	Complexidade Temporal	Justificação
132	`Map<Station, Integer> indexOf = new HashMap<>();`	**O(1)**	Criação de HashMap vazio
133-135	`for (int i = 0; i < vertices.size(); i++) {`	**O(n)**	Itera sobre todos os vértices
134	`indexOf.put(vertices.get(i), i);`	**O(1)**	Inserção em HashMap (amortizado)

Complexidade Temporal Total:

.: $O(1 + n \cdot 1) = \mathbf{O(n)}$

---

void computeSingleSourceCentralities(Graph<Station, Weight> graph, ArrayList vertices, Map<Station, Integer> indexOf, int source, double[ ] betweenness, double[ ] harmonic)

Linha	Código	Complexidade Temporal	Justificação
142	`int n = vertices.size();`	**O(1)**	Acesso a campo
143	`Deque<Integer> stack = new ArrayDeque<>();`	**O(1)**	Criação de ArrayDeque vazio

Linha	Código	Complexidade Temporal	Justificação
144	`List<List<Integer>> predecessors = initializePredecessors(n);`	**O(n)**	Ver análise do método `initializePredecessors()`
145	`double[] sigma = new double[n];`	**O(n)**	Inicialização de array
146	`double[] dist = new double[n];`	**O(n)**	Inicialização de array
148	`runDijkstraWithBrandes(...);`	**O(m log n)**	Ver análise do método `runDijkstraWithBrandes()`
149	`computeHarmonicCloseness(source, dist, harmonic, n);`	**O(n)**	Ver análise do método `computeHarmonicCloseness()`
150	`computeBetweennessFromStack(...);`	**O(m)**	Ver análise do método `computeBetweennessFromStack()`

Complexidade Temporal Total:

.: $O(1 + 1 + n + n + n + m \log n + n + m)$ = **$O(m \log n + n)$**

List<List> initializePredecessors(int n)

Linha	Código	Complexidade Temporal	Justificação
154	`List<List<Integer>> predecessors = new ArrayList<>(n);`	**O(n)**	Criação de ArrayList com capacidade n
155-157	`for (int i = 0; i < n; i++) {`	**O(n)**	Itera sobre todos os vértices
156	`predecessors.add(new ArrayList<>());`	**O(1)**	Adiciona à lista (amortizado)

Complexidade Temporal Total:

.: $O(n + n \cdot 1)$ = **$O(n)$**

---

void runDijkstraWithBrandes(Graph<Station, Weight> graph, ArrayList vertices, Map<Station, Integer> indexOf, int source, Deque stack, List<List> predecessors, double[ ] sigma, double[ ] dist)

Linha	Código	Complexidade Temporal	Justificação
165	`Arrays.fill(dist, Double.POSITIVE_INFINITY);`	**O(n)**	Preenche array de tamanho n
166	`sigma[source] = 1.0;`	**O(1)**	Atribuição
167	`dist[source] = 0.0;`	**O(1)**	Atribuição
169	`PriorityQueue<NodeDistance> queue = new PriorityQueue<>(...);`	**O(1)**	Criação de PriorityQueue vazia
170	`queue.add(new NodeDistance(source, 0.0));`	**O(log n)**	Inserção em heap binário
172	`while (!queue.isEmpty())`	**O(n)**	No máximo n iterações (cada vértice processado uma vez)
173	`NodeDistance current = queue.poll();`	**O(log n)**	Extração do mínimo do heap
174	`if (current.distance > dist[current.index] + EPS)`	**O(1)**	Comparação aritmética
175	`continue;`	**O(1)**	Salto de iteração
177	`stack.push(current.index);`	**O(1)**	Push em ArrayDeque
179	`Collection<Edge<Station, Weight>> outgoing = graph.outgoingEdges(...);`	**O(grau(v))**	Retorna arestas incidentes ao vértice
180	`if (outgoing == null) continue;`	**O(1)**	Verificação de referência
182	`for (Edge<Station, Weight> edge : outgoing)`	**O(m)**	Total de iterações: Σ grau(v) = 2m
183	`Weight weight = edge.getWeight();`	**O(1)**	Acesso a campo
184	`if (weight == null) continue;`	**O(1)**	Verificação de referência
186	`Integer wObj = indexOf.get(edge.getVDest());`	**O(1)**	Lookup em HashMap (amortizado)
187	`if (wObj == null) continue;`	**O(1)**	Verificação de referência

Linha	Código	Complexidade Temporal	Justificação
188	`int w = wObj;`	**O(1)**	Unboxing
190	`double newDist = dist[current.index] + weight.getDistance();`	**O(1)**	Operações aritméticas
192	`if (newDist + EPS < dist[w])`	**O(1)**	Comparação
193	`dist[w] = newDist;`	**O(1)**	Atribuição
194	`queue.add(new NodeDistance(w, newDist));`	**O(log n)**	Inserção no heap
195	`sigma[w] = sigma[current.index];`	**O(1)**	Atribuição
196	`predecessors.get(w).clear();`	**O(k)**	k = número de predecessores anteriores
197	`predecessors.get(w).add(current.index);`	**O(1)**	Adiciona à lista (amortizado)
198	`else if (Math.abs(newDist - dist[w]) <= EPS)`	**O(1)**	Comparação
199	`sigma[w] += sigma[current.index];`	**O(1)**	Operação aritmética
200	`predecessors.get(w).add(current.index);`	**O(1)**	Adiciona à lista (amortizado)

Nota: Cada vértice é extraído do heap no máximo uma vez: n extrações × O(log n) = O(n log n). Cada aresta é processada no máximo uma vez: m iterações. O termo O(k) de `predecessors.clear()` soma no máximo O(m) no total.

Complexidade Temporal Total:

.: $O(n + 1 + 1 + 1 + \log n + n \cdot \log n + m \cdot (1 + \log n) + m) =$ **O(m log n)**

---

void computeHarmonicCloseness(int source, double[ ] dist, double[ ] harmonic, int n)

Linha	Código	Complexidade Temporal	Justificação
207	`for (int i = 0; i < n; i++)`	**O(n)**	Itera sobre todos os vértices

Linha	Código	Complexidade Temporal	Justificação
208	`if (i == source) continue;`	**O(1)**	Comparação
209	`if (dist[i] < Double.POSITIVE_INFINITY && dist[i] > 0)`	**O(1)**	Comparações
210	`harmonic[source] += 1.0 / dist[i];`	**O(1)**	Operações aritméticas

Complexidade Temporal Total:

.: O(n·1) = **O(n)**

---

void computeBetweennessFromStack(int source, Deque stack, List<List> predecessors, double[ ] sigma, double[ ] betweenness)

Linha	Código	Complexidade Temporal	Justificação
217	`int n = sigma.length;`	**O(1)**	Acesso a propriedade length
218	`double[] delta = new double[n];`	**O(n)**	Inicialização de array
219	`while (!stack.isEmpty())`	**O(n)**	Número de vértices no stack (vértices alcançáveis)
220	`int w = stack.pop();`	**O(1)**	Pop de ArrayDeque
221	`for (int v : predecessors.get(w))`	**O(m)**	Total de iterações: Σ
222	`if (sigma[w] == 0) continue;`	**O(1)**	Comparação
223	`double coeff = (sigma[v] / sigma[w]) * (1.0 + delta[w]);`	**O(1)**	Operações aritméticas
224	`delta[v] += coeff;`	**O(1)**	Operação aritmética
226	`if (w != source)`	**O(1)**	Comparação
227	`betweenness[w] += delta[w];`	**O(1)**	Operação aritmética

Nota: O loop interno itera sobre todos os predecessores de cada vértice. No pior caso, cada aresta pode estar no DAG de caminhos mais curtos, resultando em O(m) iterações totais.

Complexidade Temporal Total:

.: O(1 + n + n·1 + m·1) = **O(m)**

---

void updateMetricsFromArrays(ArrayList vertices, Map<Integer, NodeMetrics> metrics, double[ ] betweenness, double[ ] harmonic)

Linha	Código	Complexidade Temporal	Justificação
234	`for (int i = 0; i < vertices.size(); i++)`	**O(n)**	Itera sobre todos os vértices
235	`betweenness[i] /= 2.0;`	**O(1)**	Operação aritmética
236	`NodeMetrics nodeMetrics = metrics.get(vertices.get(i).getStationID());`	**O(1)**	Lookup em HashMap (amortizado) + acesso a campo
237	`if (nodeMetrics != null)`	**O(1)**	Verificação de referência
238	`nodeMetrics.betweenness = betweenness[i];`	**O(1)**	Atribuição
239	`nodeMetrics.harmonic = harmonic[i];`	**O(1)**	Atribuição

Complexidade Temporal Total:

.: O(n·1) = **O(n)**

---

void normalizeAndComputeHubScore(ArrayList vertices, Map<Integer, NodeMetrics> metrics, int n)

Linha	Código	Complexidade Temporal	Justificação
247-250	`double maxStrength = metrics.values().stream().mapToDouble(...).max().orElse(0.0);`	**O(n)**	Itera sobre todos os valores e encontra máximo
252	`double harmonicNormalizer = (n > 1) ? (n - 1d) : 1d;`	**O(1)**	Cálculo aritmético
254	`double betweennessNormalizer = (n > 2) ? ((n - 1d) * (n - 2d) / 2d) : 1d;`	**O(1)**	Cálculo aritmético

Linha	Código	Complexidade Temporal	Justificação
256	`for (Station station : vertices) {`	**O(n)**	Itera sobre todos os vértices
257	`NodeMetrics nodeMetrics = metrics.get(station.getStationID());`	**O(1)**	Lookup em HashMap (amortizado)
258	`if (nodeMetrics == null) continue;`	**O(1)**	Verificação de referência
260	`double betwNorm = ...`	**O(1)**	Operações aritméticas
261	`double harmonicNorm = ...`	**O(1)**	Operações aritméticas
262	`double averageStrength = ...`	**O(1)**	Operações aritméticas
263	`double strengthNorm = ...`	**O(1)**	Operações aritméticas
265-267	Chamadas a `clamp01()` e atribuições	**O(1)**	Operações constantes
268-271	Cálculo de `hubScore`	**O(1)**	Operações aritméticas

Complexidade Temporal Total:

.: $O(n + 1 + 1 + n \cdot 1) = \mathbf{O(n)}$

List buildScores(Collection stations, Map<Integer, NodeMetrics> metrics)

Linha	Código	Complexidade Temporal	Justificação
277	`List<StationHubScore> scores = new ArrayList<>();`	**O(1)**	Criação de ArrayList vazio
278	`for (Station station : stations)`	**O(n)**	Itera sobre todos os vértices
279	`NodeMetrics nodeMetrics = metrics.get(station.getStationID());`	**O(1)**	Lookup em HashMap (amortizado)

Linha	Código	Complexidade Temporal	Justificação
280	`if (nodeMetrics == null) continue;`	**O(1)**	Verificação de referência
282-289	`scores.add(new StationHubScore(...));`	**O(1)**	Adição à lista (amortizado) + criação de objeto

Complexidade Temporal Total:

.: O(1 + n·1) = **O(n)**

---

double clamp01(double value)

Linha	Código	Complexidade Temporal	Justificação
294-297	Operações condicionais e retornos	**O(1)**	Operações constantes

Complexidade Temporal Total:

.: **O(1)**

Considerações Finais

A análise de complexidade temporal do método `analyzeNetwork` revela que a operação mais custosa é o cálculo das centralidades utilizando o algoritmo de Brandes com Dijkstra executado para cada vértice como origem, resultando em uma complexidade total de **O(n·m log n + n^2)**. Este valor é crucial para entender o desempenho do sistema ao lidar com grandes redes ferroviárias, onde o número de estações e conexões pode ser significativo. A eficiência do algoritmo impacta diretamente na capacidade de análise e otimização da infraestrutura ferroviária.